

Transaction Management Application

Overview

This project is a transaction management backend application built with Spring Boot.

The application provides APIs to:

- Create transactions
 - Retrieve transactions by account
 - Filter transactions by status
 - Reverse posted transactions
 - Persist transaction data safely using JSON storage
 - Maintain high-performance in-memory access using ConcurrentHashMap
 - Persist updates asynchronously using a write buffer
 - Provide structured application, audit, and error logging
-

Technology Stack

Backend

- Java 21
- Spring Boot
- Spring Web
- Jackson ObjectMapper
- Lombok
- SLF4J + Logback
- Maven

Data Storage

The application uses JSON file persistence instead of a database.

The design uses:

- Internal JSON file bundled with the application as initial data
- External JSON file used for runtime updates

Runtime data is always written outside the application package because application resources are read-only after deployment.

Project Structure

```
src/main/java
├── com.neo
│   ├── controller
│   │   └── TransactionController
│   ├── service
│   │   ├── TransactionService
│   │   └── TransactionPersistenceService
│   ├── repository
│   │   └── TransactionRepository
│   ├── model
│   ├── exception
│   └── configuration
```

Requirements

Before running the application, install:

- Java 21
- Maven 3.8+
- Node.js and npm (only required for frontend)

Verify:

```
java -version
```

```
mvn -version
```

Backend Setup

1. Clone the project

```
git clone <repository-url>
```

```
cd transaction-app
```

2. Configure application properties

Example:

```
application.properties
```

```
spring.application.name=transaction-viewer-backend
```

```
server.port=8080
```

```
#this property is used by the flush method to determine the scheduled delay  
to execute the flush method
```

```
app.persistence.flush-delay=1000
```

```
#enable virtual threads
```

```
spring.threads.virtual.enabled=true
```

```
# Path (relative to the working directory the app is run from) where the
```

```
# JSON data file lives. Seeded automatically from  
src/main/resources/data/transactions.json
```

```
# on first run if it doesn't already exist.
```

```
app.data.externalFile=D://transactions.json
```

```
app.data.internalFile=classpath:data/transactions.json
```

```
# Pretty-print JSON responses for easier manual inspection during development
```

```
spring.jackson.serialization.indent-output=true
```

```
#log path property
```

```
LOG_PATH=/opt/transaction-app/logs
```

Dependencies

The main Maven dependencies required are:

Spring Web

Provides REST API support.

Jackson Databind

Used for JSON serialization/deserialization.

Jackson Java Time Module

Required for Java 8 date/time types:

```
<dependency>
  <groupId>com.fasterxml.jackson.datatype</groupId>
  <artifactId>jackson-datatype-jsr310</artifactId>
</dependency>
```

Lombok

Used for reducing boilerplate code.

Running the Backend

using Maven (you must be in the backend root folder in CMD):

```
mvn spring-boot:run
```

Runtime Data Initialization

During startup:

1. The application checks if the external JSON file exists.
2. If it does not exist:
 - Loads the internal resource JSON.
 - Copies initial transactions into the external file.
 - Loads transactions into memory.
3. If it exists:
 - Loads transactions directly from the external file.

The external file becomes the source of truth during runtime.

Persistence Design

Transactions are stored in memory:

`ConcurrentHashMap<String, Transaction>`

For faster account searches:

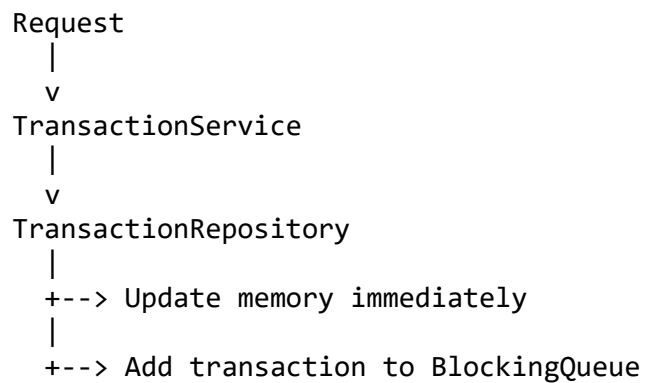
`ConcurrentHashMap<String, Set<String>>`

where:

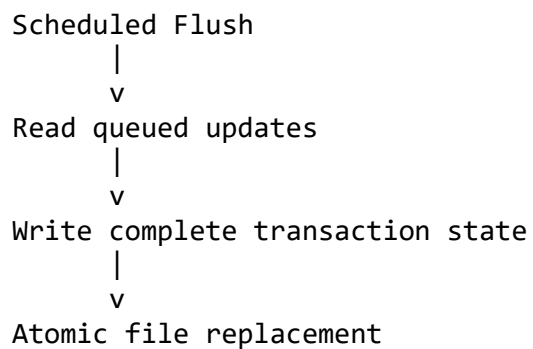
- key = account ID
 - value = transaction IDs
-

Persistence Flow

When creating or updating a transaction:



Every second:



Logging

The application uses three log files:

application.log

Contains technical application events:

- Startup
- Loading data
- Memory operations
- Persistence operations

transactions.log

Contains business audit events:

Examples:

```
CREATE_TRANSACTION  
REVERSE_TRANSACTION
```

errors.log

Contains unexpected failures:

Examples:

- File write failures
 - Runtime exceptions
 - Internal errors
-

API Examples

Create Transaction

POST /api/transactions

Creates a transaction with:

status=PENDING

Retrieve Transactions

GET /api/transactions/{accountId}

Optional filter:

?status=POSTED

Reverse Transaction

PATCH /api/transactions/{transactionId}/reverse

Rules:

- Only POSTED transactions can be reversed.
 - Other states return an error response.
-

Frontend

The frontend provides:

- Transaction listing
- Status filtering
- Transaction creation
- Transaction reversal

Run frontend (you must be in the frontend root folder in CMD):

```
npm install
```

```
npm run dev
```

AI Usage Summary

See:

PROMPTS.md

for detailed AI prompts and generated results.